CSA06 – Design and Analysis of Algorithms

CO1 Assessment Tool 1: Scalability Audit of ShopSphere

Student Name: _____ Registration No.: _____

Objective

Analyze the asymptotic efficiency of five ShopSphere back-end modules, derive the required counts and recurrences, estimate their behavior at n = 1,000,000, and recommend redesigns. The question specifies a server rate of 10■ basic operations per second.

(a) Understand the Problem

Module	Input size and dominant operation	Expected growth
M1 Duplicate detector	Compare each pair of product records (title and seller).	Quadratic: Θ(n²)
M2 API rate limiter	Audit each request in windows of sizes 1, 2, 4, … below n.	Linear geometric sum: Θ(n)
M3 S1 sequential search	Compare the search key against each catalogue entry.	Best Θ(1); average/worst Θ(n)
M3 S2 binary search	Compare with the middle key and discard half the sorted index.	Θ(log n)
M4 recommendation engine	Recursive subproblem work plus linear merge.	Current Θ(n²); redesign Θ(n log n)
M5 sales analytics	Two recursive halves plus n log n work per level.	Θ(n log² n)

(b) Analyse the Iterative Modules

M1. The inner loop compares product i with every later product. The total is (n–1)+(n–2)+…+2+1 = Σ(k=1 to n–1) k = n(n–1)/2. Thus the exact comparison count is n(n–1)/2 and the running time is Θ(n²). The nested loops use Θ(1) auxiliary space, excluding the catalogue and any duplicate-output storage.

M2. The audit counts are 1+2+4+…+2■, where 2■ is the largest power of two strictly less than n. The geometric sum is 2^(k+1)–1. Equivalently, for integer n>1 the count is 2^ceil(log■ n)–1. Since this value is less than 2n, the tight bound is Θ(n), not Θ(n log n). For n=1,024: 1+2+…+512 = 1,023 audits.

(c) Solve S2 by Backward Substitution

```
T(n) = T(n/2) + 1
     = [T(n/4) + 1] + 1 = T(n/4) + 2
     = T(n/8) + 3
     = ... = T(n/2^k) + k
Stop when n/2^k = 1, so k = log■ n (for n a power of 2).
T(n) = T(1) + log■ n = 1 + log■ n.
Therefore T(n) = Θ(log n).
```

Iterative binary search uses Θ(1) auxiliary space. Recursive binary search uses Θ(log n) call-stack space in the worst case.

(d) Apply the Master Theorem

M4 current: T(n)=4T(n/2)+n. Here a=4, b=2, f(n)=n, and n^(log_b a)=n². Since f(n)=O(n^(2–ε)) for ε=1, Master Theorem Case 1 gives Θ(n²). Doubling n increases the dominant work by about 4×.

M4 redesign: T(n)=2T(n/2)+n. Here a=2, b=2, f(n)=n, and n^(log_b a)=n. Case 2 gives Θ(n log n). Doubling n changes the dominant term by about 2·log■(2n)/log■(n), which is slightly more than 2× and approaches 2× for large n.

M5: T(n)=2T(n/2)+n log n. Here a=2, b=2, and n^(log_b a)=n. The three basic cases in their simplest forms do not directly capture the extra log factor. Extended Master Theorem Case 2 (or a recursion tree) gives Θ(n log² n): at each level, the total work is Θ(n log(n/2^j)); summing across Θ(log n) levels gives Θ(n log² n).

(e) Best, Average and Worst Search Cases

S1 sequential search: Best case: product is first, 1 comparison. Worst case: product is last or absent, n comparisons. If present, assume each of n positions is equally likely, so the conditional average is (n+1)/2. If absent, there are n comparisons. If p is the probability the product exists, E[C] = p(n+1)/2 + (1–p)n. For n=1,000,000 and p=0.5, E[C] = 0.5(1,000,001/2)+0.5(1,000,000)=750,000.25 comparisons.

S2 binary search: Best case: the key is at the middle, 1 comparison. Average and worst cases are Θ(log n); exact counts depend on the key position and tree shape. Under the recurrence model supplied in the question, T(n)=1+log■ n for powers of two.

(f) Prove f(n)=Θ(n²) and not O(n)

Given f(n)=3n²+10n log■n+50. For n≥1, log■n≤n, so n log■n≤n² and 50≤50n². Therefore f(n)≤3n²+10n²+50n²=63n². Also f(n)≥3n². Choose c■=3, c■=63 and n■=1; then 3n²≤f(n)≤63n² for all n≥1, proving f(n)=Θ(n²).

To show f(n) is not O(n), suppose f(n)≤cn for all n≥n■. But f(n)≥3n², so this would require 3n²≤cn, or 3n≤c, which fails for sufficiently large n. Hence f(n)∉O(n).

(g) Scalability Decision at n=1,000,000

Module	Operation estimate	Estimated time at 10■ ops/s
M1	499,999,500,000	4,999.995 s ≈ 83.33 min
M2	1,048,575	0.0105 s
M3 S2	≈ 21 comparisons (logarithmic)	≈ 0.00000021 s
M4 current	≈ 2n² ≈ 2×10¹² (model estimate)	≈ 20,000 s ≈ 333.3 min
M4 redesign	≈ n(log■n+1) ≈ 2.09×10■	≈ 0.21 s
M5	≈ 0.5n(log■n)² ≈ 1.99×10■ (leading-term estimate)	≈ 2.0 s

A 10-minute window permits 600×10■ = 60,000,000,000 basic operations under the stated idealized model. M1 exceeds the window and should be redesigned using a hash-based grouping by (seller, normalized title), sorting, or a database uniqueness/index strategy. The current M4 design also exceeds the window; remove replicated recursive work as proposed. M2, S2, redesigned M4, and M5 fit the estimated window. Real timings may differ because recurrence constants, memory behavior, storage, and other overhead are not specified.

Program Implementation and Output Verification

The attached Python program computes M1 and M2 counts, estimates S2 comparisons, estimates recurrence-based work for M4 and M5, evaluates S1 expected comparisons, and prints the 10-minute operation budget. Run it using Python 3 with: python main.py. No third-party libraries are required.

Source-code screenshot

main.py — source code (part 1)

```
"""
CSA06 — Design and Analysis of Algorithms
CO1_AT1: Scalability Audit of ShopSphere
Run: python main.py
"""
import math

OPS_PER_SECOND = 10**8
N = 1_000_000

def m1_comparisons(n):
    return n * (n - 1) // 2

def m2_audits(n):
    # 1 takes values 1, 2, 4, ... while i < n
    return 2 ** math.ceil(math.log2(n)) - 1 if n > 1 else 0

def binary_search_comparisons(n):
    # Recurrence model T(n)=T(n/2)+1; approximate integer bound
    return math.ceil(math.log2(n)) + 1 if n > 1 else 1

def estimated_seconds(operations):
    return operations / OPS_PER_SECOND

def fmt_time(seconds):
    if seconds < 1:
        return f"{seconds:.6f} seconds"
    return f"{seconds:.2f} seconds ({seconds/60:.2f} minutes)"

def main():
    n = N
    m1 = m1_comparisons(n)
    m2 = m2_audits(n)
    s2 = binary_search_comparisons(n)

    # Asymptotic leading-term estimates; exact constants depend on implementation.
    m4_current = 2 * n * n - n      # recurrence with T(1)=1 for powers of two; leading estimate
    m4_redesign = n * (math.log2(n) + 1)
    m5 = 0.5 * n * (math.log2(n) ** 2) + 0.5 * n * math.log2(n) + n

    print("ShopSphere Scalability Audit")
    print(f"Input size n = {n:,}")
    print(f"Server rate = {OPS_PER_SECOND:,} basic operations/second\n")
    rows = [
        ("M1 Duplicate detector", m1),
        ("M2 API rate limiter", m2),
        ("M3 S2 binary search", s2),
        ("M4 current recommendation engine (estimate)", m4_current),
        ("M4 redesigned recommendation engine (estimate)", m4_redesign),
        ("M5 sales analytics engine (estimate)", m5),
    ]
    for name, ops in rows:
        seconds = estimated_seconds(ops)
        print(f"{name}: {ops:,.0f} operations; estimated time {fmt_time(seconds)}")
```

Executed output screenshot

```
Terminal — executed program output
```

```
ShopSphere Scalability Audit
Input size n = 1,000,000
Server rate = 100,000,000 basic operations/second

M1 Duplicate detector: 499,999,500,000 operations; estimated time 4999.99 seconds (83.33 minutes)
M2 API rate limiter: 1,048,575 operations; estimated time 0.010486 seconds
M3 S2 binary search: 21 operations; estimated time 0.000000 seconds
M4 current recommendation engine (estimate): 1,999,999,000,000 operations; estimated time 19999.99 seconds (333.33 minutes)
M4 redesigned recommendation engine (estimate): 20,931,569 operations; estimated time 0.209316 seconds
M5 sales analytics engine (estimate): 209,599,497 operations; estimated time 2.10 seconds (0.03 minutes)

M2 verification for n=1,024:
Audits = 1 + 2 + 4 + ... + 512 = 1,023

S1 expected comparisons for n=1,000,000 and p=0.5:
E[C] = p(n+1)/2 + (1-p)n = 750,000.25

Note: M4/M5 operation totals are model-based estimates; constants and implementation details affect real timings.
A 10-minute window allows at most 600 * 10^8 = 60,000,000,000 basic operations.
```

Conclusion

The audit identifies quadratic pairwise comparison in M1 and the current replicated M4 as the main scalability risks. M2 is linear despite the teammate's claim, binary search is logarithmic, the redesigned M4 is $\Theta(n \log n)$, and M5 is $\Theta(n \log^2 n)$. Redesign M1 and adopt the non-replicated M4 recurrence before catalogue growth.